AgriAgent Week 1 Dossier

30 September – 5 October 2026 · what was built, from the Git history

AgriAgent Final Year Project · 11 October 2026

> **How this was written.** This dossier was rebuilt on 11 October 2026 from the Git commit history, not from earlier dossiers. Earlier Week 1 documents (several versions) overstated what was verified and have been replaced by this one. Status labels: REAL executed with real models, MOCK executed with fake model outputs, PARTIAL code exists but not run end to end.

1 Goal of Week 1

Turn an empty scaffold into a pipeline skeleton where each of the four tracks has a working, testable piece: vision (Track 1), agronomy (Track 2), data (Track 3), and the user interface (Track 4).

2 Starting point

The repository was created on 16 August 2026 from **SmartDesk**, an unrelated productivity assistant (Gmail, calendar, Telegram agents). On 3 October the SmartDesk agents were stripped out and the shared state object (`src/state.py`, `GraphState`) was rewritten for agriculture: it now carries the image, boxes, masks, spray map and prescription between pipeline stages. SmartDesk is not part of the current runtime.

3 What each track delivered

3.1 Track 1 · Core vision and safety (Chaitanya)

Delivered	Status
`src/vision/grounding_dino.py`: Grounding DINO-tiny wrapper. Text prompt in, boxes out. Has a mock mode for tests.	PARTIAL
`src/vision/sam2_wrapper.py`: SAM 2-tiny wrapper. Boxes in, masks out. Supports a corrective point prompt.	PARTIAL
`src/vision/refinement.py`: chooses a corrective point when a mask's predicted quality is below 0.85.	PARTIAL
`src/workflow.py`: the orchestrator. Runs detect → segment → refine → spray map → triage in order.	PARTIAL
Kaggle notebook (`notebooks/agriAgentShared.ipynb`) and `scripts/run_live_pipeline.py` for running real weights on a GPU.	REAL
First triage router (an external package called Laya). Replaced in Week 2.	removed

Real run, 3 October. The notebook ran Grounding DINO + SAM 2 with real weights on a Kaggle T4 GPU on a cotton field image and a synthetic image. Output figures are in `docs/images/`. This is the only real model execution in the project so far. Its result is discussed in the Week 2 dossier.

3.2 Track 2 · Agronomy reasoning (Sumant)

Delivered	Status

`src/agronomy/vlm_critic.py`: asks a vision-language model to name the weed and crop stage, returns structured JSON.	MOCK
`src/agronomy/knowledge_store.py`: searchable store over the ICAR-DWR herbicide bulletin (`data/knowledge_base/`), using ChromaDB, with an offline in-memory fallback.	PARTIAL
`src/agronomy/prescription.py`: turns a weed + crop stage into a herbicide, dose and safety notes, checking banned chemicals and dose ceilings.	PARTIAL
`src/agronomy/system2_node.py`: plugs diagnosis + prescription into the workflow as the "System 2" step.	MOCK

3.3 Track 3 · Data and evaluation (Sahil)

Delivered	Status
`data/download_datasets.sh` and `data/datasets/DATASETS_LINKS.md`: links and a downloader for Sugar Beets 2016, DeepWeeds, CottonWeedDet and others.	PARTIAL
`tests/fixtures/sample/`: 20 sugar-beet images (Sugar Beets and Weeds dataset) plus an annotation file, used by tests.	REAL data

No datasets are stored in the repository; they must be downloaded.

3.4 Track 4 · UI and integration (Amit)

Delivered	Status
`app/app.py` and `app/components/`: Streamlit dashboard. Upload an image, run the pipeline, see overlays and KPIs, download GeoJSON.	MOCK
`src/vision/visualization.py`: draws boxes and mask overlays.	PARTIAL
`.github/workflows/ci.yml`: GitHub Actions running lint and tests on every push (19 tests at the end of Week 1).	REAL

4 What Week 1 did not do

- No accuracy was measured against ground truth.
- The agronomy step never called a real language model.
- Nobody used the Streamlit app with real model weights.
- The weed-versus-crop question (does the spray map avoid the crop?) was not examined.

5 Lessons

1. Multiple AI-written dossiers described Week 1 as "complete" and "verified". They described intent, not evidence. From Week 2 on, every claim carries a status label.
2. Working from a copied, unrelated codebase (SmartDesk) left confusing leftovers that took time to remove.
3. The pieces were built in parallel and tested separately; the hard work is joining them with real models.